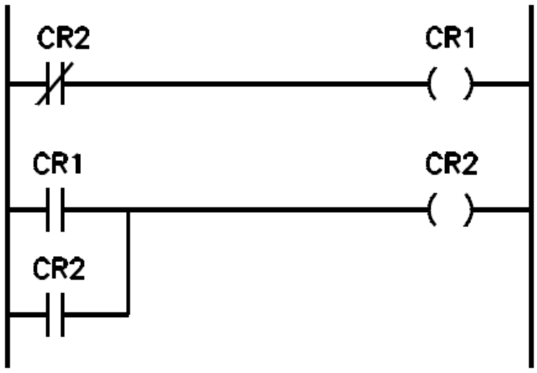
Selected programming issues - LD language

- Single time operations
- Logical operations, shifts and rotations
- Sequential control

1

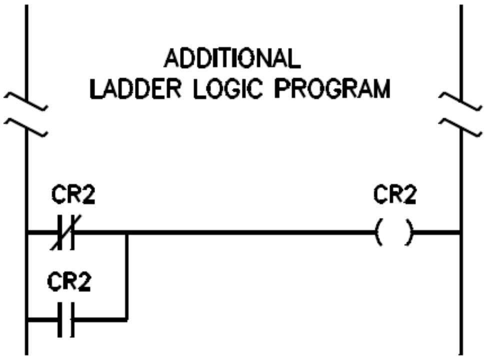
One Shot in first cycle



In the first cycle CR1=1, then CR1=0

2

One Shot in first cycle

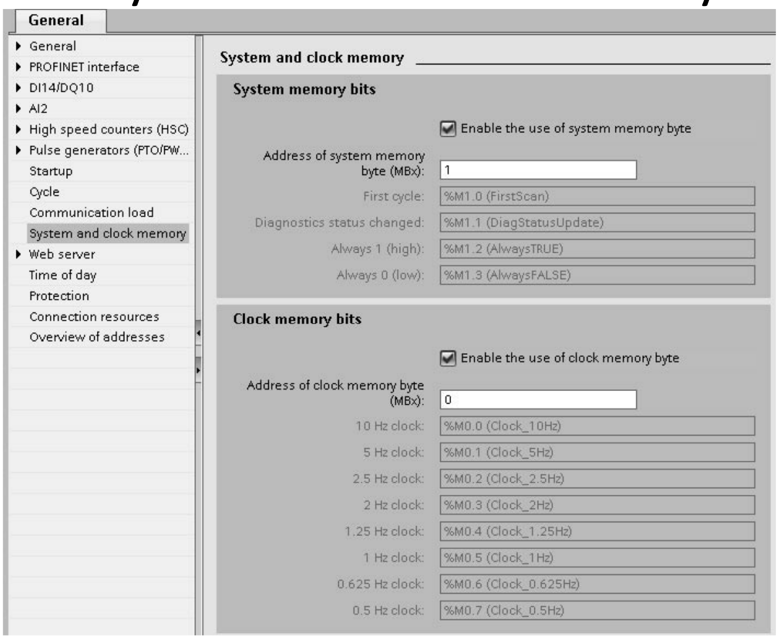


Placed at the end of the program

In the first cycle CR2=0, then CR1=1

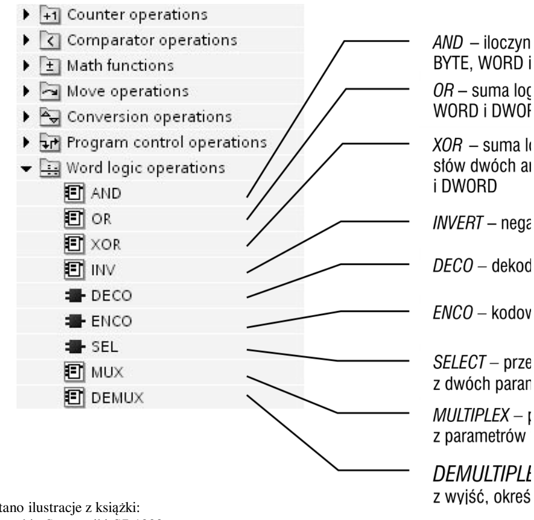
3

System and clock memory



4

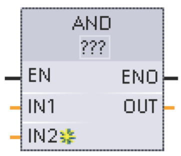
Logical operations on bytes, words and double words



Wykorzystano ilustracje z książki:
J. Kwaśniewski „Sterowniki S7-1200 w praktyce inżynierskiej"

5

AND, OR, XOR, INV

LAD / FBD	SCL
	`out := in1 AND in2;`
	`out := in1 OR in2;`
	`out := in1 XOR in2;`

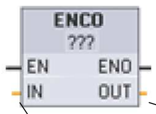
ENO is always TRUE

IN1	16#59CA	0	1	0	1	1	0	0	1	1	1	0	0	1	0	1	0
IN2	16#8559	1	0	0	0	0	1	0	1	0	1	0	1	1	0	0	1
AND	16#0148	0	0	0	0	0	0	0	1	0	1	0	0	1	0	0	0
OR	16#DDDB	1	1	0	1	1	1	0	1	1	1	0	1	1	0	1	1
XOR	16#D393	1	1	0	1	1	1	0	1	1	0	0	1	0	0	1	1

Bit Negation

6

Coding and decoding




```
out := ENCO(_in_);
```

Returns the least significant
bit equal to 1 position
If there is no such bit, ENO=0=0

0000 0000 0000 0000	0 i EN=0
1011 0000 0000 0001	0
0000 1111 0000 1000	3
0000 0000 1010 0000	5
0001 0000 0100 0000	6
1010 0001 1110 0000	5

7

Coding and decoding

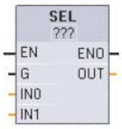

```
out := DECO(_in_);
```

Returns a value with
bit set(IN % MAX)

DECO IN value			DECO OUT value (Decode single bit position)
Byte OUT	Min. IN	0	00000001
8 bits	Max. IN	7	10000000
Word OUT	Min. IN	0	0000000000000001
16 bits	Max. IN	15	1000000000000000
DWord OUT	Min. IN	0	00000000000000000000000000000001
32 bits	Max. IN	31	10000000000000000000000000000000

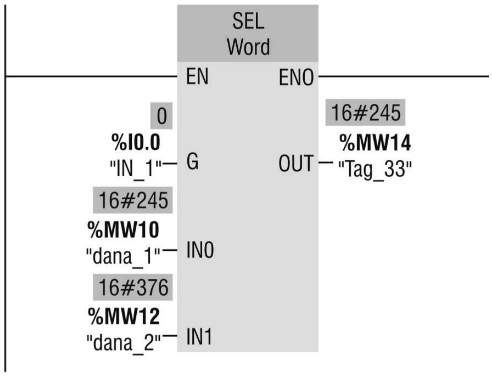
8

2-input multiplexer




```
out := SEL(
    g:=_bool_in,
    in0:-_variant_in,
    in1:=_variant_in);
```

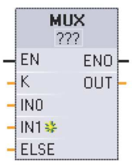
Returns the IN0 or IN1 value, depending on the key G



Wykorzystano ilustracje z książki:
J. Kwaśniewski „Sterowniki S7-1200
w praktyce inżynierskiej"

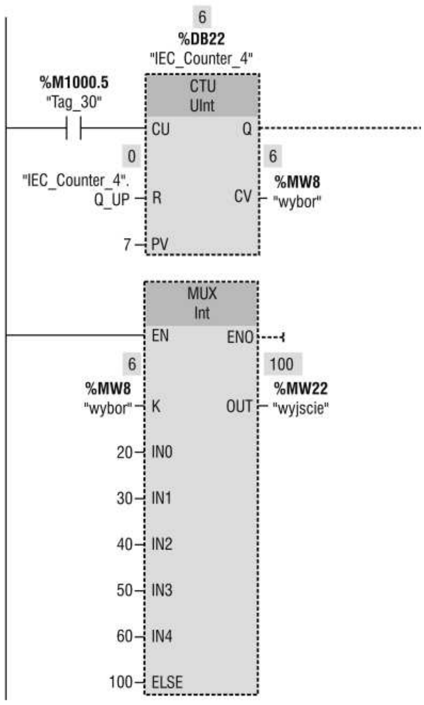
9

multiplexer




```
out := MUX(
    k:=_unit_in,
    in1:=variant_in,
    in2:=variant_in,

[...in32:=variant_in,]
    inelse:=variant_in);
```

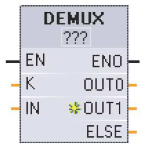
Returns one of the IN0…Ini values, depending on the key K,



Wykorzystano ilustracje z książki:
J. Kwaśniewski „Sterowniki S7-1200
w praktyce inżynierskiej"

10

Demultiplexer



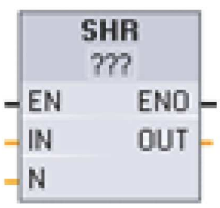

```
DEMUX(
    k:=_unit_in,
    in:=variant_in,
    out1:=variant_in,
    out2:=variant_in,

[...out32:=variant_in,]

outelse:=variant in);
```

Returns the IN value
to one of the outputs
depending on the key K

Wykorzystano ilustracje z książki:
J. Kwaśniewski „Sterowniki S7-1200
w praktyce inżynierskiej"

11

Shifts and rotations

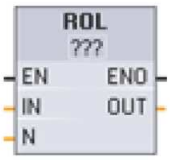



```
out := SHR(
    in:=_variant_in_,
    n:=_uint_in);
out := SHL(
    in:=_variant_in_,
    n:=_uint_in);
```

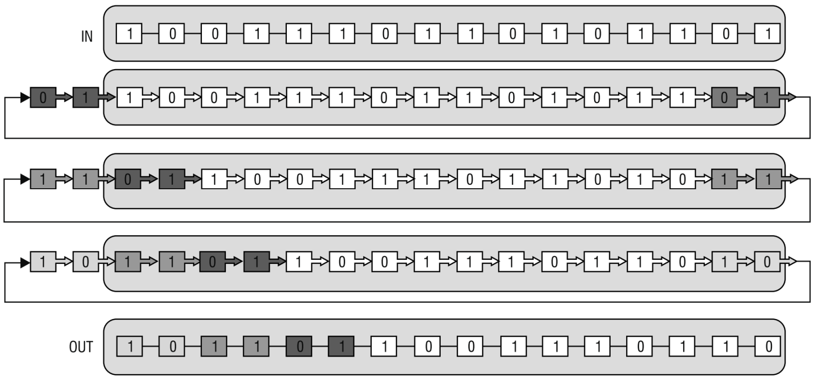
Shift the bits of a Word to the left by inserting zeroes from the right (N = 1)			
IN	1110 0010 1010 1101	OUT value before first shift:	1110 0010 1010 1101
		After first shift left:	1100 0101 0101 1010
		After second shift left:	1000 1010 1011 0100
		After third shift left:	0001 0101 0110 1000

12

Shifts and rotations

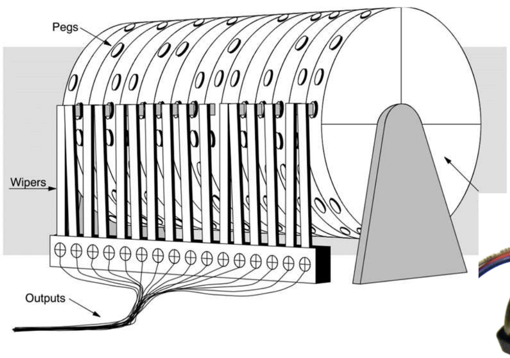


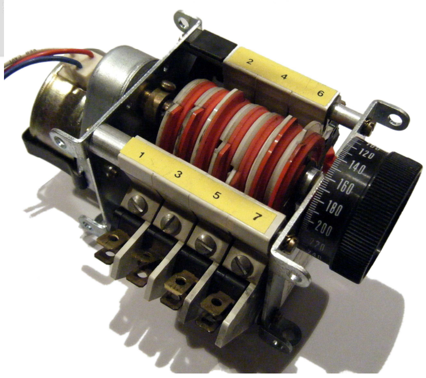

```
out := ROL(
    in:=_variant_in_,
    n:=_uint_in);
out := ROR(
    in:=_variant_in_,
    n:=_uint_in);
```



13

Drum Control Techniques

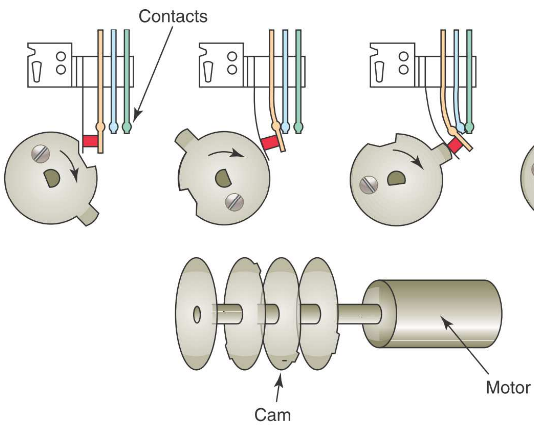




The **cam timer** or **drum sequencer**

14

Drum Control Techniques

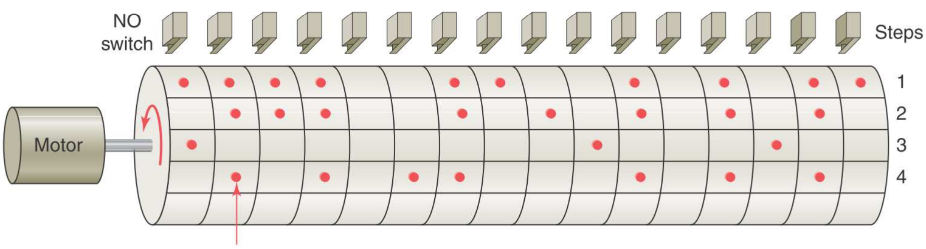


15

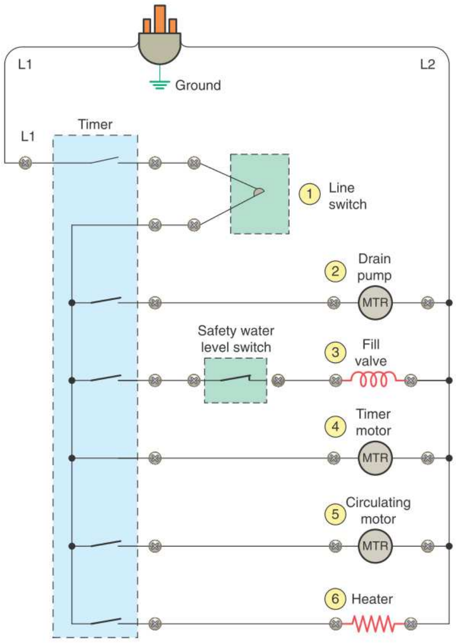
Drum Control Techniques

Equivalent sequencer data table

0	1	0	1	0	1	1	0	0	0	1	0	1	0	1	0	4
1	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	3
0	1	1	1	0	0	1	0	1	0	1	0	1	0	1	0	2
1	1	1	1	0	0	1	1	0	0	1	0	1	0	1	1	1



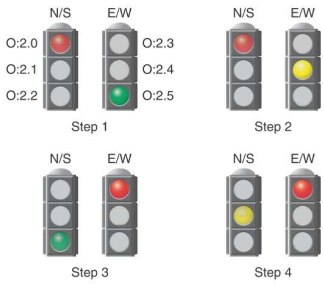
16



Machine function		Timer increment	Active devices
Off		0–1	
First prerinse	Drain Fill Rinse Drain	2 3 4–5 6	1 2 4 1 3 4 5 1 4 5 6 1 2 4 5
Prewash	Fill Wash Drain	7 8–10 11	1 3 4 5 1 4 5 6 1 2 4 5
Second prerinse	Fill Rinse Drain	12 13–15 16	1 3 4 5 1 4 5 6 1 2 4
Wash	Fill Wash Drain	17 18–30 31	1 3 4 1 4 5 6 1 2 4 5
First rinse	Fill Rinse Drain	32 33–34 35	1 3 4 5 1 4 5 6 1 2 4 5
Second rinse	Fill Rinse Drain	36 37–41 42	1 3 4 5 1 4 5 6 1 2 4 5
Dry	Dry Drain Dry	43–58 59 60	1 4 6 1 2 4 6 1 4 6

Figure 12-5 Dishwasher wiring diagram and timing chart.

17



		15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Output word →	O:2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
																		Positions
Sequencer file	B3:0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Start
	B3:1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1	Step 1
	B3:2	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	Step 2
	B3:3	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	Step 3
	B3:4	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	Step 4

Figure 12-8 Four-step sequencer.

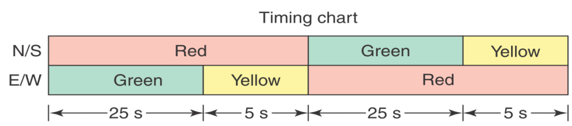


Figure 12-12 Time-driven sequencer output program.

18

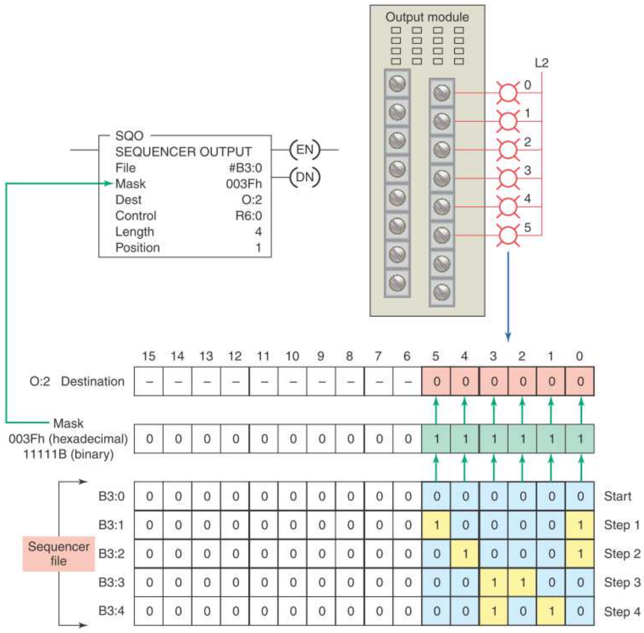
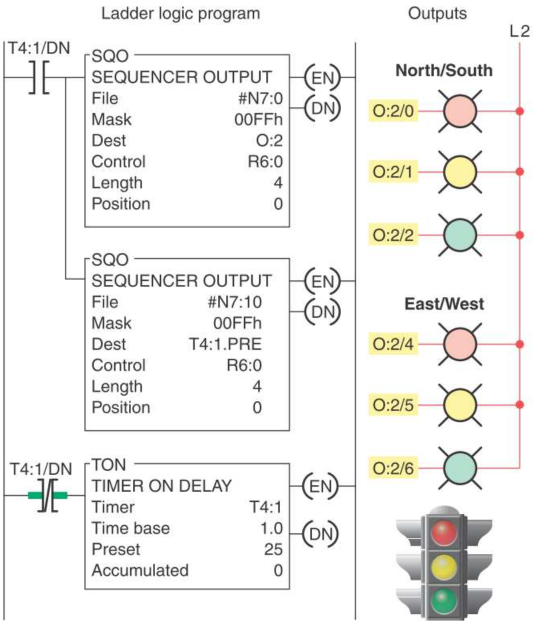
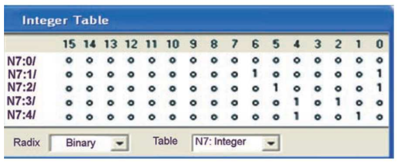


Figure 12-9 Sequencer moving data through a mask word.

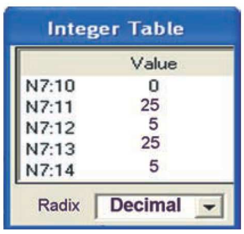
19

Time-driven sequencer





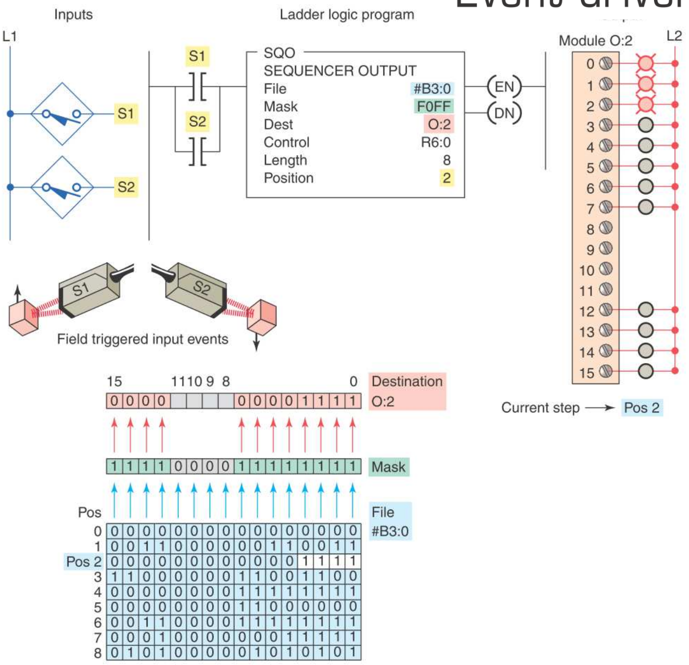
igure 12-13 Sequencer file #N7:0 light cycle settings.



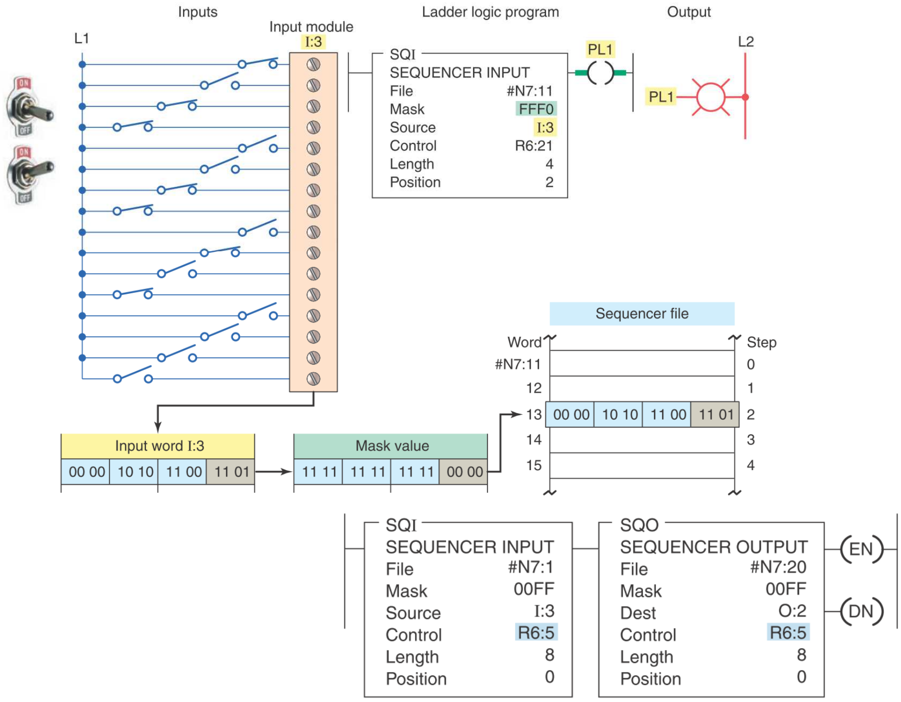
Sequencer file #N7:10 timer settings.

20

Event-driven sequencer



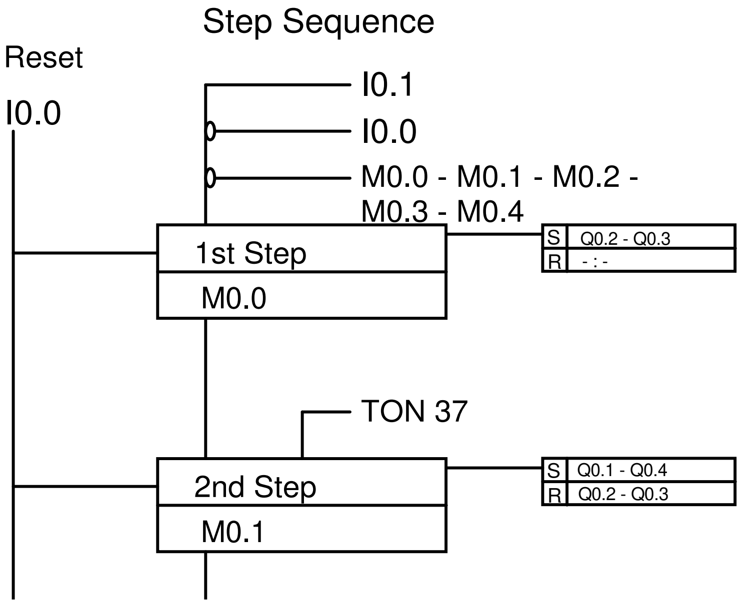
21



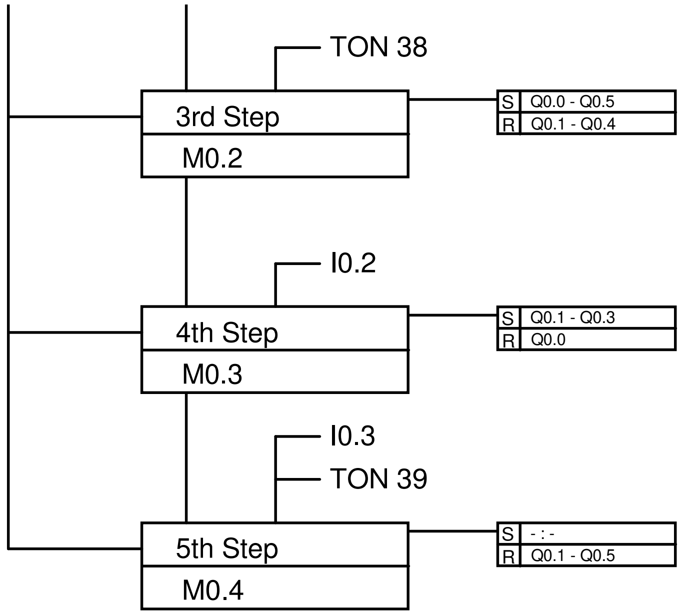
22

Drum Control Techniques – LD implementation

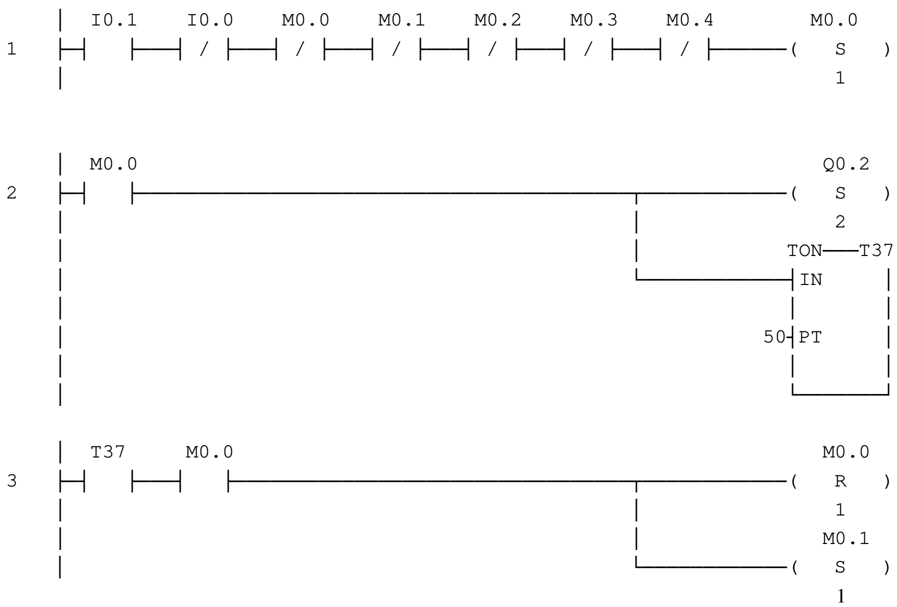
23



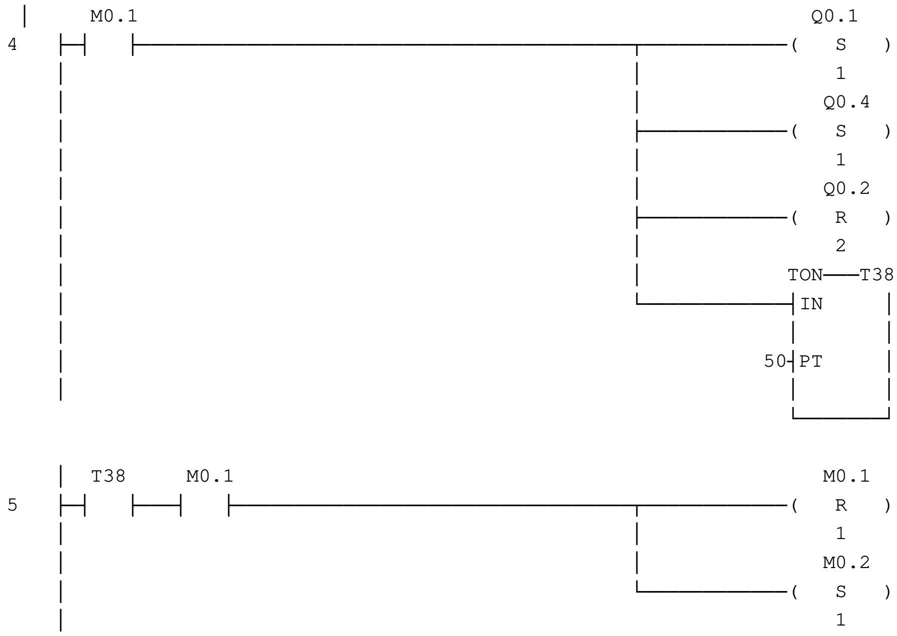
24


TON 38
3rd Step
M0.2
S Q0.0 - Q0.5
R Q0.1 - Q0.4
I0.2
4th Step
M0.3
S Q0.1 - Q0.3
R Q0.0
I0.3
TON 39
5th Step
M0.4
S - : -
R Q0.1 - Q0.5

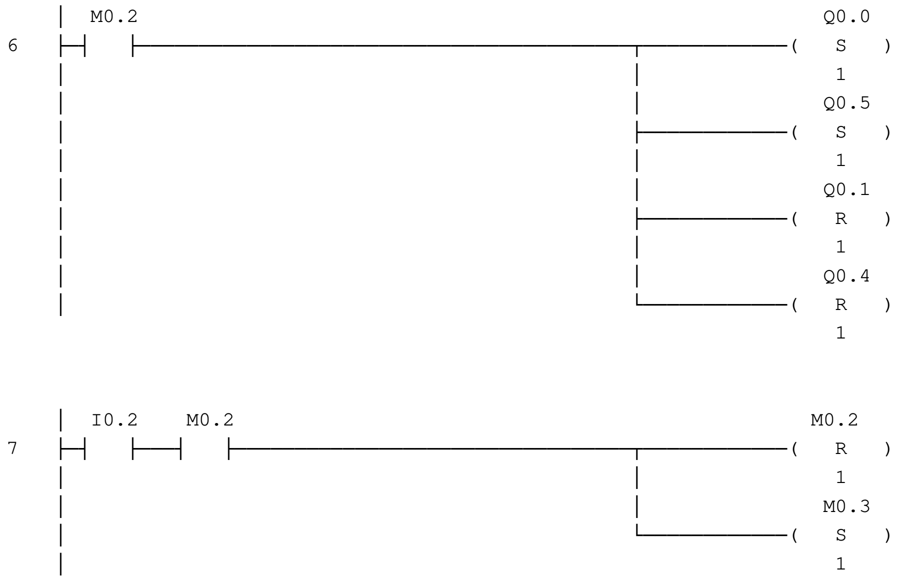
25


1
I0.1
I0.0
M0.0
M0.1
M0.2
M0.3
M0.4
M0.0
S
1
2
M0.0
Q0.2
S
2
TON
T37
IN
50
PT
3
T37
M0.0
M0.0
R
1
M0.1
S
1

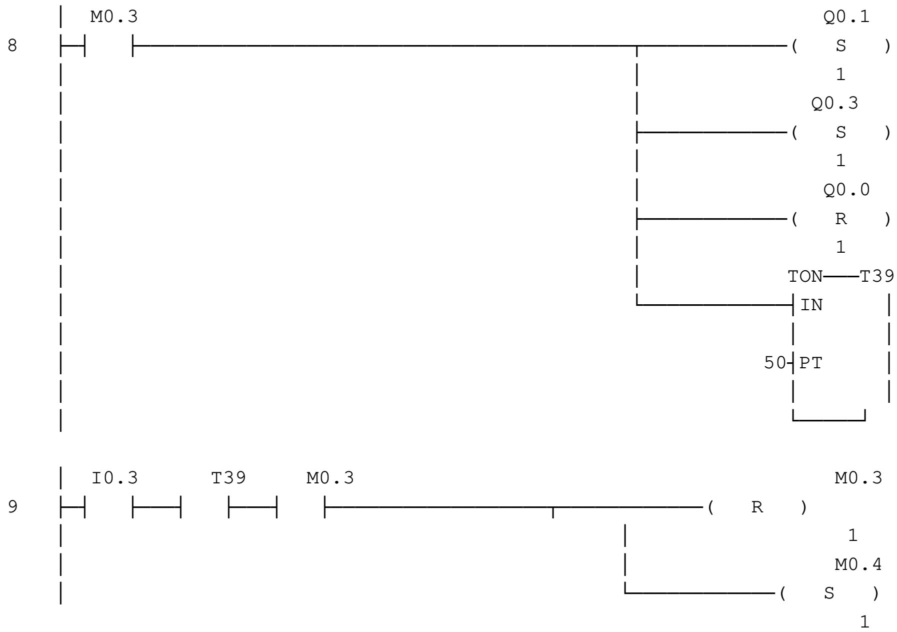
26


4
M0.1
Q0.1
S
1
Q0.4
S
1
Q0.2
R
2
TON
T38
IN
50
PT
5
T38
M0.1
M0.1
R
1
M0.2
S
1

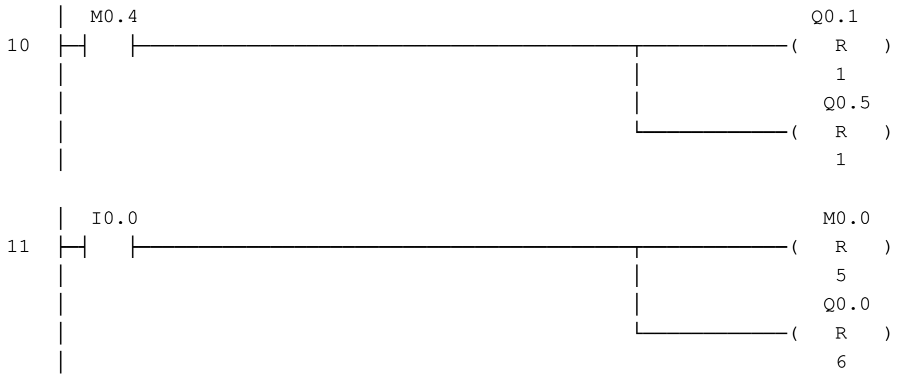
27


6
M0.2
Q0.0
S
1
Q0.5
S
1
Q0.1
R
1
Q0.4
R
1
7
I0.2
M0.2
M0.2
R
1
M0.3
S
1

28


8
M0.3
Q0.1
(S)
1
Q0.3
(S)
1
Q0.0
(R)
1
TON T39
IN
50 PT
9
I0.3
T39
M0.3
M0.3
(R)
1
M0.4
(S)
1

29


10
M0.4
Q0.1
(R)
1
Q0.5
(R)
1
11
I0.0
M0.0
(R)
5
Q0.0
(R)
6

30